

Contents

1	VarBlockSpMM: A Shape-Aware CUDA Kernel for Packed Variable-Block Sparse–Dense Matrix Multiplication	1
1.1	Abstract	1
1.2	1. Introduction	2
1.3	2. Background and Related Work	2
1.4	3. Data Representation and Semantics	3
1.5	4. Kernel Design	3
1.5.1	4.1 Row ownership	3
1.5.2	4.2 Cooperative staging	3
1.5.3	4.3 Shape-aware asynchronous pipeline	3
1.6	5. Baselines and Experimental Method	4
1.7	6. Results	4
1.7.1	6.1 End-to-end regime sweep	4
1.7.2	6.2 Profiling evidence	5
1.8	7. Discussion	5
1.9	8. Threats to Validity and Limitations	6
1.10	9. Reproducibility	6
1.11	10. Conclusion	6
1.12	References	7

1 VarBlockSpMM: A Shape-Aware CUDA Kernel for Packed Variable-Block Sparse–Dense Matrix Multiplication

Cohen

Project research manuscript, 21 August 2026

1.1 Abstract

Sparse–dense matrix multiplication (SpMM) is commonly implemented using scalar sparse formats or fixed-size block formats. Both choices are imperfect when a matrix naturally consists of dense blocks whose row and column dimensions vary independently: scalar expansion discards block structure, while global padding introduces storage and bandwidth overhead. This paper presents **VarBlockSpMM**, an FP32 CUDA implementation of the non-transpose product $C = AB$ for a packed variable-block sparse row (VBSR) matrix A and dense column-major panels B and C . Block heights and widths independently take values in $\{8, 16, \dots, 64\}$, and no global block-size padding is required.

The implementation combines row-owned output assignment, right-hand-side-width specialization, per-thread instruction-level parallelism, cooperative staging of reusable B tiles, and a measured row-shape dispatch. For 32-column panels, tall rows use asynchronous double buffering while short rows retain a lower-overhead single-buffer kernel. The direct path requires no atomics or execute-time temporary workspace. It is evaluated against a persistent scalar-CSR cuSPARSE baseline and a persistent slot-grouped cuBLAS baseline over 128 synthetic regimes on one NVIDIA GeForce RTX 5060 Ti. In the checked-in short sweep, the final direct method is fastest in all 128 cases. Relative to the earlier four-accumulator direct kernel, successive wider-ILP and staging changes provide a $1.46\times$ geometric-mean speedup over the complete grid; the final RHS-32 double-buffer refinement adds $1.11\times$ over its 32-case slice. Profiling links the improvement to reduced redundant loads and reduced long-scoreboard stalls. These results establish a strong single-device proof of concept, but not a general performance claim: multi-architecture testing, real application matrices, repeated-seed experiments, and stronger statistical reporting remain necessary.

1.2 1. Introduction

Many hierarchical, finite-element, graph, and scientific operators contain small dense submatrices embedded in a sparse global structure. When these submatrices share a single size, block sparse row (BSR) storage exposes useful locality. Real partitions, however, often yield blocks with different heights and widths. Converting such a matrix to scalar CSR loses explicit block structure; rounding every block to one maximum dimension wastes storage and arithmetic; and dispatching a separate dense GEMM for every block pays substantial scheduling overhead.

VarBlockSpMM studies the narrower problem

$$C = AB,$$

where $A \in \mathbb{R}^{M \times K}$ is divided into variable-height block rows and variable-width block columns, while $B \in \mathbb{R}^{K \times N}$ and $C \in \mathbb{R}^{M \times N}$ are dense column-major panels. The implementation supports FP32, non-transpose multiplication and $N \in \{8, 16, 32, 64\}$. Its design question is: **can a direct, workspace-free CUDA kernel retain the compactness of variable blocks while outperforming reasonable persistent library compositions across a controlled range of irregularity, degree, locality, and panel width?**

This work makes four engineering contributions:

1. A validated packed VBSR representation with independent row and column partitions and 64-bit scalar/value offsets.
2. A race-free row-owned kernel family that specializes only panel width while retaining variable block dimensions as runtime loop bounds.
3. A profiler-guided sequence of optimizations: wide accumulation to reuse each A value, cooperative shared-memory staging of B , and selective asynchronous double buffering for tall RHS-32 rows.
4. A reproducible comparison against persistent cuSPARSE and grouped-cuBLAS constructions, accompanied by correctness, sanitizer, timing, and hardware-counter evidence.

The contribution should be interpreted as an implementation and empirical case study. The associated prior-art search found no directly matching packed variable-row/variable-column FP32 SpMM interface, but it was scoped rather than exhaustive and is not a patentability or universal novelty claim.

1.3 2. Background and Related Work

GPU SpMM research has long emphasized that workload structure and row irregularity determine the useful mapping of sparse work to warps and thread blocks. Yang, Buluç, and Owens derive general GPU SpMM design principles and demonstrate row-splitting and merge-based approaches for scalar sparse matrices [1]. More recent systems exploit block structure and tensor cores, but generally target fixed or structured block layouts rather than the independently variable blocks studied here [2].

NVIDIA cuSPARSE provides SpMM for several sparse formats. Its Blocked-ELL path uses a block size belonging to the format and is tuned for fixed power-of-two blocks [3]; it does not directly encode a matrix in which every block row and column may have a different extent. NVIDIA cuBLAS provides grouped batched GEMM with per-group matrix dimensions [4]. This is a closer building block, but independent GEMMs that update the same output region cannot simply execute together without respecting accumulation dependencies. VarBlockSpMM therefore uses a slot-split organization as its cuBLAS baseline: blocks at the same ordinal position in different block rows form non-overlapping grouped calls, and slots execute sequentially with accumulation into C .

The immediate motivating application is adaptive construction of hierarchical H^2 matrices. Boukaram et al. describe a non-uniform batched BSR product and split it into a bounded number of non-uniform batched GEMM kernels in the absence of a directly suitable GPU implementation [5]. VarBlockSpMM targets the underlying packed variable-block operation rather than the full hierarchical construction algorithm.

Asynchronous global-to-shared copies are a natural mechanism for hiding the latency of reusable dense-panel tiles. NVIDIA documents that these operations decouple transfer initiation from completion and can

overlap data movement with computation; hardware support is available from compute capability 8.0 for suitably aligned small copies [6]. The final RHS-32 tall-row kernel applies this mechanism conservatively, after profiling showed that a synchronous staged design had shifted its principal cost from long-scoreboard stalls toward block barriers.

1.4 3. Data Representation and Semantics

Let block-row boundaries be $r_0 = 0 < r_1 < \dots < r_R = M$ and block-column boundaries be $c_0 = 0 < c_1 < \dots < c_S = K$. Block row i has height $h_i = r_{i+1} - r_i$; block column j has width $w_j = c_{j+1} - c_j$. Every stored block A_{ij} is a dense $h_i \times w_j$ column-major array. Its contribution is

$$C[r_i : r_{i+1}, :] += A_{ij}B[c_j : c_{j+1}, :].$$

The host format stores block-row pointers, column-block indices, scalar row/column offsets, value offsets, and tightly packed values. Validation checks monotonic offsets, legal dimensions, in-range indices, and agreement between metadata and the value buffer. Device ownership is separated from execution planning: **Matrix** owns immutable GPU storage, while **Plan** contains a non-owning view and immutable row-classification metadata. Consequently, the matrix must outlive its plan.

Packing each block at its actual dimensions avoids the capacity cost of padding all blocks to 64×64 . The tradeoff is that address generation and loop trip counts remain dynamic inside the kernel. The format uses 64-bit scalar and value offsets to avoid limiting aggregate matrix or packed-value size through 32-bit addressing metadata.

1.5 4. Kernel Design

1.5.1 4.1 Row ownership

One cooperative thread array (CTA) owns one block row, and its threads cover disjoint elements of that row’s output panel. Because no two CTAs update the same output scalar, the kernel needs neither atomic operations nor partial-output reduction storage. Each CTA iterates through the blocks of its row and accumulates their contributions before writing its owned elements.

The dispatch specializes on N , not on the runtime block dimensions. RHS 8 uses a scalar mapping with one output element per thread. For RHS 16 and 32, a thread accumulates eight output columns for one local row; RHS 64 uses sixteen. Reusing one loaded A value across multiple independent accumulators reduces repeated sparse-value traffic and exposes independent fused multiply-add instructions, at the cost of registers and reduced thread-level parallelism. Measurements rejected wider mappings where that tradeoff became unfavorable.

1.5.2 4.2 Cooperative staging

For RHS 32 and 64, all local output rows in a CTA consume the same $w_j \times N$ slice of B . Direct global loading redundantly requests these values for each local row. The staged kernels cooperatively load the active B slice into shared memory once per block, then reuse it across local rows. A shared leading dimension of 65 floats separates neighboring columns and avoids the worst power-of-two bank-alignment pattern. The largest logical RHS-64 tile contains 64×64 floats, or 16 KiB.

The compiled synchronous staged kernels use 9,344 bytes of shared memory and 56 registers per thread for RHS 32, and 17,664 bytes and 64 registers per thread for RHS 64, without local-memory spills. RHS 16 remains on direct global loads because its smaller reuse opportunity did not repay synchronization overhead.

1.5.3 4.3 Shape-aware asynchronous pipeline

RHS-32 rows are classified once during matrix construction. Rows of at most 16 scalars use a 128-thread single-buffer CTA. Taller rows use 256 threads and two shared tiles. While the CTA computes on tile t ,

per-thread asynchronous copies initiate loading of tile $t + 1$. A pipeline wait and CTA barrier at the block boundary establish visibility and prevent buffer reuse before computation finishes.

Splitting a mixed-height matrix into two kernel launches is enabled only when its mean block degree is at least eight. At lower degree, measurements showed that the second launch costs more than shape separation saves; homogeneous matrices already require only their applicable launch. The final tall-row RHS-32 kernel uses 48 registers per thread and 17,664 bytes of shared memory; its 128-thread companion uses 54 registers and 9,344 bytes.

The same policy is deliberately not applied to RHS 64. Two full RHS-64 tiles require roughly 34 KiB per CTA. In the representative 4,096-row case, that allocation reduced residency and regressed median time from 2.891 ms to 4.419 ms. The accepted design therefore retains the single-buffer RHS-64 kernel.

1.6 5. Baselines and Experimental Method

Two persistent baselines separate the proposed representation from readily available library strategies.

Scalar CSR cuSPARSE. Dense blocks are expanded once into scalar CSR. The plan owns its CSR arrays, descriptors, library handle, and workspace, all of which are reused across executions.

Slot-grouped cuBLAS. Packed blocks are grouped by identical (h_i, w_j) shape within each nonzero slot. One `cublasSgemmGroupedBatched` call is issued per slot; outputs within a call do not overlap, and later slots use $\beta = 1$ to accumulate. Device pointer arrays for changing B and C addresses are refreshed on the execution stream and this required marshaling is included in timing.

The checked-in grid contains 128 workloads: 1,024 block rows and columns; mean degrees $\{2, 4, 8, 16\}$; RHS widths $\{8, 16, 32, 64\}$; four block-size distributions (uniform, low variance, high variance, and bimodal); and local or random column selection. All runs use seed 1. Each short-sweep point records seven measured iterations after three warmups. GPU-event median and p95 times are reported separately from synchronized host median and p95 times. Construction and host-to-device input transfer are excluded.

The environment was an NVIDIA GeForce RTX 5060 Ti 16 GB, CUDA 13.4.46, driver 610.57.01 with CUDA UMD 13.3, MSVC 19.44, Release configuration, native architecture, and fast FP32 math. Profiler-instrumented durations are not mixed with unprofiled benchmark timings.

Correctness is checked against a double-accumulating CPU reference using a 64-case matrix spanning all RHS widths, distributions, degrees $\{1, 4, 8, 16\}$, both locality patterns, and non-default streams. The checked-in Release test passed on 21 August 2026. The project also records successful memcheck and racecheck runs with zero findings.

1.7 6. Results

1.7.1 6.1 End-to-end regime sweep

The final hybrid direct path is fastest among the four tested methods in all 128 checked-in workloads. This win count is descriptive of this grid and device, not evidence that the method dominates arbitrary matrices or GPUs.

The optimization sequence is summarized below. Geometric means treat every grid point equally and are not weighted to represent a production workload distribution.

Change	Compared with	Evaluated slice	Geometric-mean median speedup	Additional observation
Wider per-thread ILP	Four-accumulator direct kernel	All 128 cases	1.24×	RHS 16/32/64: 1.21×/1.33×/1.43×

Change	Compared with	Evaluated slice	Geometric-mean median speedup	Additional observation
Cooperative B staging	Wider-ILP kernel	All 128 cases	$1.18\times$	Every RHS-32 and RHS-64 case improved
Wider ILP + staging	Four-accumulator direct kernel	All 128 cases	$1.46\times$	Direct path retained 128/128 wins
RHS-32 double buffer + shape dispatch	Single-buffer staged kernel	32 RHS-32 cases	$1.11\times$	All degree-8 and degree-16 cases improved

Longer representative measurements provide scale beyond the short sweep:

4,096-row workload	Final/accepted kernel	Comparison	Median result
Bimodal, random, degree 16, RHS 64	2.895 ms staged	4.887 ms wider-ILP	$1.69\times$ faster
Bimodal, random, degree 16, RHS 32	0.468 ms double-buffered	0.549 ms staged	$1.17\times$ faster

For the RHS-64 workload, the earlier four-accumulator result was 8.430 ms; the cumulative progression to 2.895 ms is $2.91\times$. These representative figures answer an optimization question, but they should not substitute for full long-duration reruns of every regime.

1.7.2 6.2 Profiling evidence

Profiling of the original difficult RHS-64/high-degree kernel showed 99.54% achieved occupancy but only 0.48 eligible warps per scheduler per cycle; 71.5% of scheduler cycles had no eligible warp, and long-scoreboard waits accounted for 90.1% of the average issue interval. DRAM throughput was 56.93% of peak. Thus the limiting symptom was load latency rather than insufficient occupancy, register spilling, shared-memory spilling, or saturation of peak bandwidth.

Wider accumulation reduced redundant A traffic and raised useful work per load. Cooperative staging then reduced the representative long-scoreboard share from 78.0% to 37.1%, increased eligible warps per scheduler from 0.41 to 0.75, and raised achieved occupancy from 49.6% to 64.7%. Barrier waiting became the dominant residual cost at 36.4% of the issue interval.

For the final RHS-32 tall-row kernel, Nsight Compute confirms native asynchronous global-to-shared loads. The replay-instrumented pass reports 80.4% achieved occupancy, 1.42 eligible warps per scheduler, 47.3% of cycles with no eligible warp, 81.1% SM throughput, and 68.1% DRAM throughput. These counters support the proposed mechanism, but replay-instrumented time is excluded from benchmark speedups.

1.8 7. Discussion

The results illustrate three broader lessons. First, retaining block structure can be valuable even when block dimensions are not compile-time constants. Specializing a small orthogonal dimension—the dense panel width—captures much of the benefit without a combinatorial family of block-shape kernels. Second, occupancy alone did not diagnose the slow kernel: high occupancy coexisted with few eligible warps and dominant dependency stalls. Third, an optimization that is sound in isolation need not be universal. Double buffering helped tall RHS-32 rows, harmed RHS 64 through shared-memory residency pressure, and was not worth a second launch for low-degree mixed shapes.

The direct approach also has operational advantages. It owns outputs by row, so it avoids floating-point atomics and their scheduling-dependent accumulation order. It allocates no execute-time workspace, and `execute` remains asynchronous on a caller-provided CUDA stream. Conversely, plans rely on immutable matrix metadata, support only four panel widths, and currently expose only FP32 non-transpose operation. These choices make the implementation focused rather than general-purpose.

1.9 8. Threats to Validity and Limitations

The principal limitations are experimental.

- **One GPU and software stack.** Results may change with memory hierarchy, shared-memory capacity, scheduler behavior, library versions, or architecture.
- **Synthetic matrices.** The generators control block-size distribution and column locality, but they do not establish performance on matrices from a real application.
- **One seed and short timing runs.** Seven measured repetitions after three warmups are suitable for a development regime map, not publication-grade uncertainty estimates. The grid lacks confidence intervals, repeated process launches, and multi-seed aggregation.
- **Restricted operation.** Only FP32, non-transpose A , column-major dense panels, block dimensions 8–64 in steps of eight, and RHS widths 8–64 are supported.
- **Baseline scope.** The baselines are credible persistent library compositions, but they are not an exhaustive comparison with research kernels, alternative reorderings, custom fixed-block ensembles, or application-specific approaches.
- **Useful-GFLOP accounting.** Reported throughput counts arithmetic from stored dense blocks. It is useful for comparisons within this study but does not measure semantic sparsity inside a nominally dense stored block.
- **Novelty scope.** Failure to identify a directly matching implementation in a scoped search is not proof that none exists.

Accordingly, the defensible conclusion is that VarBlockSpMM is a well-supported single-GPU prototype whose design wins the tested regime map—not that it is universally faster than cuSPARSE or cuBLAS.

1.10 9. Reproducibility

The repository requires CMake 3.25 or newer, CUDA 13.4, and an MSVC host compiler with C++26 draft support. On Windows:

```
scripts\build.ps1
build\Release\vbsr_benchmark.exe --rows 4096 --degree 8 --rhs 32 --distribution high --locality random
scripts\run_grid.ps1 -Rows 4096 -Reps 20 -Warmup 5
```

The complete short sweeps are stored in `data/regime_map.csv`, `data/regime_map_wide_ilp.csv`, `data/regime_map_staged.csv`, and `data/regime_map_double_buffered.csv`. Nsight Compute exports are retained separately under `data/`, and exact profiling commands are documented in `docs/profiling.md`. A stronger replication should run at least 20 measured iterations after five warmups as already supported by the grid script, repeat each configuration across independent process launches and seeds, report bootstrap confidence intervals, and test multiple GPU architectures.

1.11 10. Conclusion

VarBlockSpMM demonstrates that packed variable-block SpMM can be mapped efficiently to CUDA without global block padding, atomic output updates, or execute-time temporary storage. The central mechanism is not a single universal kernel but a restrained, measured policy: specialize dense-panel width; reuse sparse values through independent accumulators; stage dense-panel tiles only when reuse justifies synchronization; and double-buffer only the row shapes and degrees that repay its resource cost. On the tested RTX 5060 Ti grid, the final implementation beats its scalar direct predecessor, persistent scalar-CSR cuSPARSE, and

slot-grouped cuBLAS in every case. The next scientific step is not further headline optimization, but broader validation across hardware, application matrices, seeds, and statistically rigorous runs.

1.12 References

- [1] C. Yang, A. Buluç, and J. D. Owens, “Design Principles for Sparse Matrix Multiplication on the GPU,” *Euro-Par 2018*, 2018. <https://arxiv.org/abs/1803.08601>
- [2] G. Li et al., “High Performance Unstructured SpMM Computation Using Tensor Cores,” 2024. <https://arxiv.org/abs/2408.11551>
- [3] NVIDIA, “cuSPARSE Generic API: `cusparseSpMM` and Blocked-ELL,” CUDA Toolkit Documentation. <https://docs.nvidia.com/cuda/cusparse/>
- [4] NVIDIA, “cuBLAS `cublas<t>gemmGroupedBatched`,” CUDA Toolkit Documentation. <https://docs.nvidia.com/cuda/cublas/#cublas-t-gemmgroupedbatched>
- [5] W. Boukaram, Y. Liu, P. Ghysels, and X. S. Li, “Adaptive Sketching Based Construction of H^2 Matrices on GPUs,” *IPDPSW 2025*, 2025. <https://arxiv.org/abs/2506.16759>
- [6] NVIDIA, “Asynchronous Data Copies,” *CUDA Programming Guide*. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html>